Stochastic Expectation Models & the Topological Score Grammar

Fifteen design questions answered before implementation begins — with binding guidelines, merge-tree validation findings, and a metric-to-grammar-to-codebase mapping table

The Question (verbatim)

1.i.

During all changes moving forward have all changes be strictly additive, so: a new score_governor.py that reads tracker state (same read-only relationship as lava_bridge.py), maintains role elections, runs the four rule modules, and emits a ScoreState — phrase boundary flags, clock parameters, prominence ranking, tension scalar, current pitch field. voice_manager._compute_targets then consumes ScoreState the same way it consumes freq_offset_hz: as another summed/multiplied layer before smoothing and send. Routing already has the right shape — add governor-derived sources (TENSION, PROMINENCE_RANK, PHRASE_POSITION, BEAT_PHASE) to the Source enum and let the existing routing matrix carry them, which means the performer can rewire score-linguistic governance live from the EDIT surface.

1.ii.

Also, while implementing changes/additions, write the code base such that it will be easy to incorporate new material, methodologies, and capabilities in the future. Implement additions with the foresight for a degree of 'modularity' that will allow for easy expansion and development in the future and with a generality that will not limit, restrict, or predefine what developments will be possible and how we will be able to incorporate them.

2.

Consider this a longer range goal/targeted vision: what principles and theories, broadly, from physics, information theory, and statistics do we have for defining a dynamic stochastic system expectation model. Could conditional entropy be computed, then statistical expectation on entropy progression; what other statistical attributes of the stochastic system could be used to derive/define a meaningful statistical expectation model that would elegantly tie into the stochastic score and topological grammar theory? Importantly — consider this a core question we would like to develop and understand: how could we use the statistical and entropic information of the individual blobs, all blobs taken together, and the entire stochastic system to build a predictive and generative model for score evolution / score composition and progression? How/what potential information could be used to inform methodologies that compute, predict, and use the likelihood and statistical expectation of the stochastic system, and importantly, what options are there and how would we define a measurable and computable stochastic state of the system, and on what elements of the system would that entropic notion be dependent (such as but not limited to the physical/kinematic properties of the system, geometric/topological developments, individual blob and swarm behaviors) to generate subsequent states in the score in a manner that impacts/guides generated score grammar attributes and global or individual voices and likelihood of subsequent score states, voice leading, harmony, dissonance, etc. Similarly, what are possible definitions, methodologies, and frameworks defined by physically or mathematically computable attributes, system states, and composite global entropies that could serve as solution to Meyer's thesis (tension as blocked expectation, resolution its release) and Huron's ITPRA cycle — including the design hooks of a second, perceptual tension definition alongside the physical one; boundary schedule (how far in advance boundaries are made audible-as-coming); and the Xenakian option of the time-span layer governing density distributions. Lastly, how would we specifically define these musical notions and how would we implement them musically, and based off of what?

3.i.

Provide prospective definitions and ways we could implement a robust and wide range of pattern language layers: each blob owns a rhythmic pattern generator rather than (or layered on) a continuous sustained voice. The obvious primitive is the Euclidean rhythm E(k, n): distribute k hits (define hits here) as evenly as possible across n equal time slots.

Parameterize by blob properties (area → k, persistence → n, x-position → rotation) with the metrical governor supplying the clock. As we will not embed Tidal itself, but rather a small pattern-transformation algebra reimplemented in Python: how specifically could we define and implement a manifestation of Tidal-esque transformations, and with what governor and governing principles could transformations be applied to voices' patterns (and in what potential ways)? How could these literal "score linguistics" manifest (patterns are the words, governor transformations are the syntax), including the SuperCollider gated/percussive mode with pattern events as timed OSC bundles? What other transformations and governings could be defined so as to ensure a robust and diverse system of pattern transformation semantics?

3.ii.

Keep in mind a main goal will be developing the schema-lexicon direction (Gjerdingen: stock two-voice contrapuntal frames, combined and ornamented according to conventions of succession) — so while implementing additions now, write the code base such that it will be easy to incorporate these later. The pattern-language layer can be lifted from purely rhythmic to schema-like: a small lexicon of pitch-rhythm frames, each blob instantiating and ornamenting one, governors controlling schema succession. Lastly, what would be considered a "small lexicon" while ensuring a robust range, and what other schema, specifically, could we consider?

4.

What are potential possibilities, theories, empirically measurable qualities of system states, or computable notions we could draw from to define concepts of organization, disorder, chaos, entropy, and tools (such as earth mover's distance or KL divergence) to define a single system instance/snapshot/summary? How could we use those notions of system stochasticity, order, and compositional summaries and the physical, topological, statistical, or geometric priors that define them to predict the likelihood of subsequent entropic states and expected macro state compositions? Importantly, how could we use these notions to define prolongational reduction — the ebb and flow of tension and relaxation — e.g., how would we define and implement analogs of dissonance building tension, consonance releasing it, and distance from the tonic, instead of Lerdahl's tonal targets (harmonic distance from a home region, dissonance level, register spread, dynamics)?

5.

Generate a testing suite to validate the accuracy and expressiveness of the merge tree computation (merge_tree.py already exists — the merge tree of the luminance field is a nested hierarchical segmentation, structurally identical to a grouping analysis). Once we validate the merge tree and ensure it suitable, we will implement attribute design principles based on the fact that the merge tree is structurally isomorphic to a Schenkerian level hierarchy — root = background, leaves = surface; depth in the tree = structural level.

6.

Explain what is meant by hysteresis in property-based election (top-k by persistence, area, or age per role — simple, legible, but causes churn when ranks swap). What other ranking approaches are there that can be non-hyperparameter dependent, e.g. percentiles or top-k? How will we address the fact that often there are few blobs, maybe three, moving slowly?

7.i.

Expand on Temperley's procedural contribution (GTTM-style preference-rule systems as global optimization via dynamic programming, computed incrementally left to right, extended to key-finding and harmony). How could we define weighted rule scoring? What is meant by "left to right", and how would we incorporate the extension to key-finding and harmony — what would that manifest as in our problem setting?

7.ii.

Provide more detail regarding the implementation template for the governor rule modules: each module maintains a small set of candidate interpretations of the recent governor event stream, scores them with weighted soft rules, and commits incrementally — which naturally supplies the hysteresis the role-election layer needs, because switching interpretations costs score. What options do we have and how would they be implemented, with what methodology?

8.

Provide potential options for defining topological election in our problem setting — specifically what prospective topological elements could be used or are available, what various elements of the score they could be associated to and

govern, how that governance would be implemented, and how it would manifest specifically in the score.

9.

What are options for dictating time-span reduction in GTTM's sense of structural importance — determining how essential a note is, and assigning heads vs. ornaments? We plan on implementing persistence-as-prominence (GUDHI H0 persistence as feature significance; rank voices; foreground/background mixer). However, as the blob lifecycle is fairly consistent, comparable, and not long-lived — a short timescale compared to a musical score — what are other options for determining time-span reduction, and how could those options be implemented and impact the score?

10.

When we begin making changes let's begin with the grouping-structure additions; but for now, suggest other events I could use from the blobs and the system as a whole. Births, deaths, and merges are fairly consistent and predictable at fairly regular intervals. How else, and what other system properties (relational attributes between blobs, system organization of all blobs) or empirical attributes (physics, geometric, or topologically based) could we use to define grouping and boundaries? How could we specifically implement a small preference-rule scorer over the governor blobs' event stream emitting phrase boundaries that trigger global gestures (collective envelope resets, filter sweeps, register shifts, texture thinning)?

11.

As it will be hard to define 'most' in "most periodic" (periodicity is somewhat uniform, short-timescale, simple): can we select periodicity attributes of the entire system, or entropy or stochastic attributes; or use the periodicity observed in one blob's movements or changing geometry; or use the relation BETWEEN two or more blobs — the periodicity of their movements with respect to one another, a ratio of geometric or physical attributes? Would it be possible, and what would the methodology be, to define behavioral-election roles using observable or computable periodicity or progression of dynamic stochastic system information, statistical development, or chaotic/entropic changes? If so, how?

12.

When we begin incorporating changes, keep in mind the handover policy: when a governor dies or is deposed, crossfade its control stream to the successor over several seconds rather than snapping (smoothed control signals, VarLag philosophy). Also: how could we use the predicted stochasticity and entropy of the system and subsequent states — an almost markovian entropy prediction — to weight and navigate the handover policy as well as the global state evolution of the score?

13.

What do you suggest to do for this: a preference-rule system scores competing structural interpretations by summing weighted soft evidence and picks the best?

14.

When we implement constraint-based composition, do not separate the governors from those that generate; there are not enough blobs in the system. Have all blobs generate, and governors selected from all blobs to ALSO narrow the space (allowed pitch field, density ceiling, register bands) — each and all voices solve locally within constraints.

15.i.

Clarify neo-Riemannian / transformational harmony for the pitch layer. Also: what options are there for selecting the governors, and what information, progression, behavior, features etc. from the governors are possible candidates to define voice-leading behavior — and what options are there for that behavior?

15.ii.

Clarify Cohn's frame (the Tonnetz; parsimonious voice-leading as the metric of distance; hexatonic/octatonic cycles as coherent but tonally unmoored progression) and Lewin's transformational attitude ("what did I do to get here?"), and the design hook of a governed Tonnetz walk (step rate and step type by tension; resolution as shortest parsimonious path home; governors emit transformations, never objects).

15.iii.

To govern harmony and harmonic drift, what other options do we have? For example, beyond a single blob governing, could we use Wasserstein distances, entropic variations, statistical expectations, etc.?

The Response

Guidelines **1.i, 1.ii, 3.ii, 10 (prefix), 12 (prefix), and 14** are logged as binding constraints for when implementation starts — strictly additive changes, modular/general design, schema-ready pattern layer, governors-also-generate, crossfaded handovers, and the specific `score_governor.py` architecture. Question 5 produced a critical empirical finding that also affects `blob_detection.py`; see its section.

2 The stochastic expectation model — the core question

Defining a computable stochastic state. The system state at time t is a layered feature vector, and the layering matters because each layer supports different entropy notions:

- *Per-blob kinematic*: position, velocity, acceleration, area, elongation, d_area — already in the tracker.
- *Relational*: pairwise distances, relative phases of vertical oscillation, a proximity graph (Delaunay or k-NN over centroids), mutual information between blob attribute streams.
- *Aggregate physical*: total kinetic energy, mean vertical flux, center-of-mass height, a velocity-coherence order parameter (mean of normalized velocity vectors — the Vicsek order parameter from flocking physics; near 1 = coherent collective motion, near 0 = disordered).
- *Topological*: the persistence diagram itself, persistence entropy (a standard TDA statistic: Shannon entropy of normalized lifetimes — low when one blob dominates, high when significance is evenly spread), Betti-0 count, merge-tree depth and balance.
- *Distributional*: the spatial occupancy histogram (coarse grid over the ROI of where wax is), and the velocity distribution.

Entropy machinery that is actually computable here. Symbolize the system: coarse-grain the aggregate state into a small alphabet (e.g., quantize [blob count, coherence, KE tercile, vertical-flux sign] into ~20–50 symbols). Then estimate an online transition matrix, and cheaply obtain: conditional entropy H(next|current) per state (how uncertain the immediate future is *from here*), the entropy rate of the chain (global unpredictability), and *surprisal* –log p(observed transition) — the instantaneous "that was unexpected" signal. Surprisal and conditional entropy are the two axes IDyOM uses for musical expectation, so applying them to the lamp's own state stream makes the physical system and the musical model share one formalism — the elegant tie-in requested. For continuous signals, permutation entropy (ordinal-pattern entropy of a time series) is robust, model-free, nearly free to compute, and works on short windows — the right tool for per-blob or aggregate signals. Transfer entropy between blob pairs gives *directional* influence (which blob's motion predicts another's) — a principled governor-election criterion (Q11). Richer but heavier: predictive information / excess entropy (how many bits the past carries about the future), Lyapunov exponent estimates, and recurrence quantification analysis (RQA: recurrence rate, determinism, laminarity from a recurrence plot — excellent for regime detection and very citable). Distances: Wasserstein/bottleneck between consecutive persistence diagrams (the canonical TDA metric — rate of topological change), Wasserstein or KL between the current occupancy histogram and the equilibrium reference (the frozen background gives equilibrium for free), Jensen-Shannon when symmetric and bounded is wanted.

Entropy progression and prediction. Track the entropy time series itself; fit an AR(1)/Kalman or exponential-smoothed trend; the *derivative* of conditional entropy is a leading indicator of regime change (entropy rises before merges/eruptions as the system leaves a well-worn region of state space). Predicted next-state entropy is exactly the "almost-Markovian entropy prediction" named in Q12.

Mapping to score generation. The clean scheme: **surprisal → tension events** (a surprising physical transition spikes dissonance/dynamics — Meyer operationalized: expectation was violated, tension results); **conditional entropy → ambiguity** (uncertain future = harmonically ambiguous, dense, unstable texture; certain future = clear triad, sparse, stable); **hazard functions → schedule** (from lifecycle statistics, estimate the probability a merge/boundary occurs in the next N seconds; rising hazard drives the audible-as-coming behaviors — accelerando, crescendo, dominant-function drift — implementing Huron's Tension phase; the event's arrival pays

the Prediction reward with consonance). The ITPRA mapping in full: Imagination = the model's predictive distribution (sonified as subtle foreshadowing material drawn from the predicted next harmonic field); Tension = hazard-weighted ramp; Prediction = surprisal at the event; Reaction = immediate accent/timbral response within ~100 ms; Appraisal = slow revaluation over the following phrase (the harmonic field actually moves). Yes — this absolutely factors into the score's methodology, and it is the strongest journal-paper claim available: *the same expectation model runs over the physical system and the musical surface, and tension is defined as their disagreement-weighted surprisal.* Prediction error between model and lamp becomes the composition's dramatic engine.

4 Snapshot summaries, order/disorder, and non-tonal prolongational reduction

A single-instant "system summary" worth standardizing now (guideline 1.ii — a dataclass with room to grow): occupancy-grid entropy (spatial disorder), persistence entropy (topological evenness), velocity coherence (kinetic order), KE total, Wasserstein-to-equilibrium (how far from rest), diagram-Wasserstein-from-previous (rate of change), blob count, merge-tree depth. That 8-vector is the macrostate; symbolizing it feeds the Markov model above; distances between summaries (plain Euclidean after normalization, or EMD on the distributional components) define "how different is now from then," which grounds both boundary detection (Q10) and form-level recurrence (the system revisiting a macrostate can trigger musical recall — a reprise).

Prolongational reduction without tonality's given definitions. Lerdahl's model needs a tonic and a chord distance; substitute computable analogs for each term:

- *Dissonance* → **psychoacoustic roughness**, directly computable and synthesizer-controllable: Plomp–Levelt/Sethares roughness is a function of the actual partial frequencies and amplitudes sounding. Controlled via detune spread, SHIMMER, noise level, and interval content. This is the key substitution — sensory dissonance needs no tonal system at all.
- *Tonic* → a **designated home region**: a home triad (or pitch field) *paired with* the physical equilibrium state. Distance-from-tonic becomes a weighted sum of parsimonious path length on the Tonnetz (harmonic distance) and Wasserstein distance from the equilibrium occupancy (physical distance). The isomorphism — the lamp far from rest ⇔ the music far from home — is the piece's central poetic and technical claim.
- *Tension* → the composite scalar: w_1·surprisal + w_2·Wasserstein-to-equilibrium + w_3·KE + w_4·conditional entropy, smoothed. It drives roughness, Tonnetz distance, register spread, and dynamics exactly where Lerdahl's tension drives their tonal equivalents.

3.i Pattern-language layer — definitions and implementation options

A *hit* = a discrete note-onset event (a trigger with velocity, duration, and pitch-degree), as opposed to the continuous sustained voice. Representation options, in increasing power: (a) binary step vector (simplest, fine for v1); (b) event list of (position, velocity, duration, degree) — supports accent and pitch, still trivially serializable; (c) Tidal's actual model, *pattern as a function from a time-span query to events* — the most composable (transformations are function composition, patterns can be infinite/aperiodic), recommended as the core abstraction with (b) as the query result type, because it satisfies guideline 1.ii: any future pattern type is just another queryable function. Concretely: a `Pattern` protocol with one method `query(span) -> [Event]`, transformations as `Pattern -> Pattern` callables, and a **registry** so new transformations are pluggable without touching core code.

Transformation vocabulary beyond the four named: `rotate`, `rev`, `fast/slow` (density), `degrade(p)` (probabilistic drop), `euclid(k,n)`, `mask` (AND with another pattern), `superimpose`, `every(n, f)` (apply f each nth cycle), `iter` (rotate progressively), `palindrome`, `chunk`, `swing(amount)`, `ply` (repeat each hit), `off(t, f)` (delayed transformed copy — instant canon), accent-pattern application, and pitch-degree ops (transpose-within-field, invert, arp). Governor bindings: tension → degrade probability (inverse) and density; BEAT_PHASE → rotation; prominence rank → velocity/ply; phrase boundary → pattern reset or a fill (`every`-triggered); harmonic field → the degree pool patterns draw from; blob merge → `superimpose` of the two blobs' patterns for a crossfade period (a beautiful, legible gesture). SC side as stated: a gated sibling of `\mpeVoice`, OSC bundles timestamped against the shared clock, with

the clock's elastic/projective behavior (Hasty) preserved because bundles are scheduled from Python, not from a rigid SC TempoClock.

3.ii Schema lexicon — noted as a design-for target

"Small" in Gjerdingen's own corpus is real: his core galant repertory is roughly 15–25 schemas, and pedagogically ~10 carried most of the weight. Here: **8–15 frames** is the right size — small enough to be learnable-by-ear across an installation visit (schemas only work if the listener starts recognizing them), large enough for succession variety. Rather than importing galant schemas literally, define schema = (melodic degree contour, bass degree contour, metric footprint in beats, tension profile). Include 3–4 *borrowed* skeletons for citability (Prinner: descending 6-5-4-3 over 4-3-2-1; Romanesca: descending bass thirds; Fonte: sequential down-a-step pair; a converging cadence) plus original frames idiomatic to the system (a "plume" schema: rising contour with accelerating rhythm mapped to blob ascent; a "merge" schema: two lines converging to a unison; a "stall": oscillation around one degree). Other galant candidates for depth: Monte, Ponte, Meyer, Do-Re-Mi, Sol-Fa-Mi, Quiescenza, Fenaroli. Succession governed by a small first-order Markov table over schema types (governors reweight the table) — *exactly* the same machinery as the state-transition model in Q2, another unification. Code implication now: the pattern `Event` type should carry an optional degree field and patterns should be groupable into multi-beat frames from day one, so schemas drop in later without rework.

5 Merge tree validation — suite delivered, and it found a critical bug

The suite (`test_merge_tree.py`, 14 tests: oracle-verified topology, analytic persistence, localization, saddle levels, nesting order, noise stability with the persistence-stability bound, monotone/gamma invariance, enrichment matching, temporal coherence, degenerate inputs, performance) runs standalone or under pytest, and includes an independent union-find **oracle** that computes ground-truth merge structure over the same filtration.

First run: 10 of 14 failed, catastrophically (29 "persistent" leaves from a single smooth Gaussian). Root cause, verified empirically: **the GUDHI dimension order is wrong.** `gudhi.CubicalComplex(top_dimensional_cells=arr.flatten(), dimensions=[sh, sw])` — GUDHI expects the fastest-varying dimension first; for a row-major flatten of an (h, w) array that is `[sw, sh]`. With the current ordering the field is read at the wrong stride — scrambled — whenever the image is not square. Swapping to `[sw, sh]` (or passing the 2D array directly, which lets GUDHI handle ordering): 0 spurious pairs, and 11/14 tests pass. **The identical pattern is in blob_detection.py line ~512.** Two possibilities: the live lamp ROI downsamples to square-ish dimensions (masking the bug), or detection has been operating on a scrambled field and the watershed/dedup stages have been partially rescuing it — which would also explain why the persistence threshold needed such counterintuitive calibration. Verify against the live tree's actual ROI dimensions; the fix is one line in each file; rerun threshold calibration after applying it.

Remaining failures after the patch, all genuine design gaps to fix before the Schenkerian reading is licensed: (1) *peak localization* — `abs(inverted - birth) < 1` with `coords[0]` grabs the first pixel anywhere at that intensity, so multiple peaks (especially equal-height ones) mislocalize or collapse; use `cc.cofaces_of_persistence_pairs()`, which returns the actual birth cell index — exact, no ambiguity. (2) *The essential class is dropped* — the dominant blob never appears as a leaf, so a one- or two-blob lamp yields an empty or internal-node-free tree; include the infinite pair with death set to the frame minimum. (3) *Internal nodes are not real saddles* — construction pairs leaves by death-sorted popping and stamps `min(child.threshold)` (a peak level) instead of the saddle level, so nesting is not guaranteed to match true topology and merge levels are wrong. The saddle level is already available (it is `death_real`, currently discarded after sorting), and the oracle in the test file is itself a correct reference implementation (union-find sweep, ~ms at downscaled resolution) that can be promoted into `merge_tree.py`. Fix these three and the root=background / leaves=surface reading is solid.

6 Hysteresis, non-hyperparameter ranking, and the few-slow-blobs problem

Hysteresis: a challenger must beat the incumbent *by a margin, for a duration* — two thresholds instead of one, so a blob whose persistence oscillates around the incumbent's does not flip the role every frame (like a thermostat that

turns on at 18° and off at 22°, never chattering at 20°). The two parameters (margin δ, dwell τ) are hyperparameters, which motivates the question. Alternatives that self-calibrate: (a) *statistical dominance* — switch only when the challenger–incumbent difference exceeds k times the observed rolling standard deviation of that difference; the threshold adapts to the signal's own noise floor. (b) *Softmax/continuous weighting* — the real answer for this system: do not elect at all; every blob holds a continuous governance weight per role, $w_i = \text{softmax}(\text{score}_i/T)$, and the role's control signal is the weight-blended sum. Election, hysteresis, and handover all dissolve into smooth weight evolution (and Q12's crossfade becomes automatic). (c) Rank-aggregation (Borda across several attributes) reduces sensitivity to any single noisy attribute. Percentiles, as noted, degenerate to ranks at n=3.

Which answers the few-blobs problem directly: with three slow blobs, discrete top-k election is nearly meaningless — continuous weights plus **system-level virtual governors** (Q11) are the design. A role can be partially carried by every blob and partially by an aggregate signal; nothing structural ever hinges on a churn-prone identity decision. This also satisfies guideline 14 naturally: since governance is a weight annotation, not a class, all blobs generate and all blobs govern to a degree.

7.i / 7.ii Temperley's DP, concretely in this setting

"Left to right" = strictly incremental in time: the analyzer never revisits committed past, it extends candidate interpretations as each new evidence frame arrives — not a limitation here but a *requirement*, since the installation runs in real time with no lookahead. Weighted rule scoring: each rule is a function $f_i(\text{evidence, candidate}) \to [0,1]$, the candidate's score is $\Sigma\, w_i \cdot f_i$ accumulated over time, and weights start hand-set (small integers, tuned by ear — Temperley's own weights were tuned by trial) with live adjustment from the EDIT surface; later they can be fit to preferred recordings, but do not block on that.

Implementation shape: each governor module keeps a **beam** of 3–5 candidate interpretations of the recent event stream. For the grouping module a candidate = a proposed boundary placement over the last window; for meter = a (tactus period, phase, grouping cardinality) triple; each incoming frame adds score to candidates it supports; a *switching cost* is charged when the committed interpretation changes — and that switching cost **is** the hysteresis, arising from the optimization itself rather than a bolted-on threshold (the precise meaning of the sentence quoted in 7.ii). Options for the machinery, cheapest first: pure beam search with additive scores (recommended start); Viterbi over a small HMM whose hidden states are the candidate interpretations and whose transition penalties encode switching cost (equivalent but more formal — better for the paper); particle filter if candidates become continuous (meter phase/period) — a natural v2 for the elastic clock.

The key-finding/harmony extension manifests directly: Temperley's key-profile scoring (score each candidate key by how well recent pitch content fits its profile) becomes *pitch-field finding* — score each candidate home triad/field by (match to recently sounded content) + (parsimonious distance from the current field) – (switching cost), committed incrementally. That gives the harmonic layer the same DP skeleton as grouping and meter: one algorithm, four instantiations — cleaner code (guideline 1.ii — one `PreferenceRuleModule` base class) and a cleaner paper.

8 Topological election — the available elements and their governance

Available topological objects, each with a natural role: **root / deep internal nodes** (the slowest, most global structure) → grouping and tension: the root's threshold drifting is a section-level signal; a new node appearing near the root is a major boundary. **Saddle nodes** (merge events at specific levels) → phrase boundaries, with boundary *strength* = the persistence of the branch being absorbed — deep saddles are section breaks, shallow saddles are motive-level articulation; a saddle can also trigger metric modulation. **Leaves** → surface/voice material, with **tree depth of each leaf = its Schenkerian level** (root-adjacent = background: slow, stable, registrally central, low roughness; deep leaves = diminution: fast patterns, ornamental degrees, higher registers) — the Q5-licensed mapping, answering "which blob gets which material" with a single number already computed. **Branch length (= persistence)** → prominence (Q9). **Tree shape statistics** — depth, balance/asymmetry (Colless-style imbalance), leaf count → texture-level parameters: a balanced tree = evenly distributed polyphony, a caterpillar tree = melody-plus-accompaniment; tree shape entropy can feed the tension composite. **Persistence-diagram coordinates** (birth, death) → register and duration assignment per voice (birth level = brightness/height of the blob's

wax → register is almost literal). **Vineyard trajectories** (the diagram points' paths over time, per the identity-carrier insight) → voice-leading: a point moving toward the diagonal is a voice fading toward release; two points converging predict a merge → pre-boundary tension. Implementation: `score_governor.py` consumes the (corrected) tree per frame, computes {leaf depth, saddle events with levels, shape stats}, and these enter the same routing-source and preference-rule machinery as everything else — no special path needed.

9 Time-span reduction beyond raw persistence

Raw persistence alone is too uniform and too short-lived to carry score-scale hierarchy. Complementary importance measures, all cheap: (a) **integrated persistence** — Σ persistence over the blob's whole life (age-weighted significance; distinguishes a briefly-bright flash from a long-standing presence even when instantaneous values match); (b) **merge-tree depth** — structural level rather than lifetime, per Q8, and independent of lifecycle duration; (c) **coupling centrality** — mutual information or transfer entropy between this blob's stream and all others: hub blobs that *drive* the system are heads (deeply ecosystemic, and reviewer-friendly); (d) **recurrence** — a re-identified blob (via the persistence-space Hungarian matching) returning after absence is head-like, the way a returning theme is; (e) **energy integral** — work done over the trajectory; and critically (f) **musical-surface criteria**, which is what GTTM's actual time-span rules use: events falling on strong beats of the governed clock and on chord tones of the current field score as heads regardless of their blob's physics. The design: prominence = smoothed weighted sum over (a)–(f), weights on the EDIT surface. This makes time-span reduction a *joint* function of physics and the score's own grammar — self-referential in exactly the Di Scipio-passing sense — and it decouples musical hierarchy from the too-short blob lifecycle: a blob's *material* can remain a head (sustained by the mixer, recalled by pattern recurrence) even across blob death and re-identification.

10 Additional grouping-boundary evidence — noted as the starting change-set

Beyond births/deaths/merges/splits (agreed: too regular to carry all segmentation), candidate boundary evidence, grouped. **Relational/configurational** — proximity-graph edge flips (nearest-neighbor topology changes), convex-hull area jumps, pairwise-distance variance shifts, collinearity/clustering events, center-of-mass crossing the ROI midline, symmetry breaks. **Statistical changepoints** — the principled general tool: Bayesian Online Changepoint Detection (BOCPD) or CUSUM run over *any* aggregate signal (KE, coherence, occupancy entropy, flux) emits boundary candidates with posterior probabilities that plug directly into the preference-rule scorer as soft evidence; this single mechanism converts every continuous signal into a boundary-evidence source and is the most future-proof choice (guideline 1.ii). **Entropic/regime** — conditional-entropy spikes, order-parameter transitions (coherent↔disordered motion), RQA laminarity shifts. **Topological** — diagram-Wasserstein between consecutive frames exceeding its rolling baseline; saddle events by depth (Q8). **Physical** — plume onsets (rapid vertical acceleration cohort), stalls (all speeds near zero — a stall ending is a strong boundary, like silence before a new section).

Scorer implementation, concretely: a typed `BoundaryEvidence` event stream; per frame, each active rule contributes score to "boundary now" — GPR2-analog: elapsed time since last event × gap salience; GPR3-analog: magnitude of attribute change normalized by rolling std; *coincidence bonus* when multiple independent evidence types co-occur within a short window (GTTM's own insight: aligned cues make strong boundaries); minus a refractory penalty proportional to closeness of the previous boundary. Threshold crossing emits `(boundary, strength)`; strength scales the global gesture (weak → filter sweep only; strong → envelope reset + register shift + texture thinning + pattern fills), all dispatched through `ScoreState` flags that `_compute_targets` and the pattern layer consume.

11 System- and relation-level periodicity for behavioral roles

Yes — and prefer it. Elect *signals*, not blobs: the meter role attaches to whichever observable exhibits the strongest periodicity, searched over a candidate bank that includes: aggregate signals (total KE, mean vertical flux, occupancy entropy — the lamp's global convection overturn is itself the master period and the most honest tactus source); **relational signals** — the relative phase between two blobs' vertical oscillations, their synchronization degree (Kuramoto order parameter over blob oscillation phases — beautiful and computable at n=3), and *period ratios*

between blob pairs, which map directly to polyrhythm and non-isochronous meter (two blobs cycling at 3:2 → a hemiola baked into the clock; the single most musically interesting option in the whole question set); and per-blob geometry oscillations (elongation breathing). Periodicity strength = autocorrelation peak height or spectral peak-to-flatness ratio, computed identically across the bank; the preference-rule/softmax machinery (Q6, Q7) then weights the bank members — so "behavioral election" generalizes cleanly from blobs to arbitrary observables with no new machinery. The erratic/tension role likewise: assign to whichever signal shows highest permutation entropy or spectral flatness. This *is* defining roles by dynamic stochastic system information — the methodology is exactly the Q2 entropy stack applied as an election criterion rather than a control signal.

12 Predictive, entropy-weighted handover — noted, with the mechanism

Logged as a binding policy (crossfade over seconds, smoothed control streams). The predictive weighting: maintain per-governor **hazard estimates** — probability of death/deposition in the next N seconds, from lifecycle statistics (a blob decelerating near the top with a neighbor converging has high merge hazard, estimable from historical event data) — and begin pre-fading *proportional to hazard* before the event, so successions are anticipated rather than reacted to. The Markov entropy prediction modulates handover character: when predicted next-state conditional entropy is **high** (succession uncertain), lengthen crossfades, distribute weight across multiple successors, and let the score's structural commitment soften (more ambiguity — musically apt, since the system itself does not know what is next); when **low**, hand over crisply and let structure firm up. Globally the same principle: governance strength ∝ model confidence. The score is decisive when the lamp is predictable and searching when it is not — that coupling is itself a strong aesthetic claim, worth a sentence in the papers.

13 Recommendation for the preference-rule scorer

Start minimal and transparent: linear weighted sums, 3–5 rules per module, log-domain scores (so evidence multiplies as probabilities would), the coincidence bonus, switching costs for hysteresis (Q7), BOCPD as one evidence source among the hand-built rules, weights exposed on the EDIT surface for live tuning by ear. Resist learning weights until the rule set stabilizes — hand-set weights are auditable, Pi 5-trivial, and match the citable lineage (GTTM/Temperley both used hand-tuned weights). One shared `PreferenceRuleModule` base class, rules as registered plugins (guideline 1.ii), instantiated four times.

14 Governors also generate — noted, and it reshapes the design correctly

Binding. Design consequence, consistent with Q6's conclusion: there is no governor *class* — governance is a set of continuous role weights carried on ordinary blob-voices, and `ScoreState` is computed from weight-blended contributions of all blobs. Constraint layers (pitch field, density ceiling, register bands) are emitted by the weighted collective and *apply to every voice including the highest-weighted contributors* — a heavily-governing blob still solves its own voice within the constraints it disproportionately shaped. Self-governance of that kind is, again, the ecosystemic loop.

15.i Neo-Riemannian clarification; governor features → voice-leading options

Clarified plainly: the harmonic layer maintains one current triad (three pitch classes). P, L, R each swap one note by a step (P: C-E-G ↔ C-E■-G; L: C-E-G ↔ B-E-G; R: C-E-G ↔ C-E-A), so consecutive triads share two notes and the third moves minimally — chord changes that sound like slow morphing rather than jumps, matching wax phenomenology. "Voices gravitate": each voice's target frequency gets a pull toward the nearest current chord tone, strength set by the governance layer.

Governor-selection options for *this* role specifically: the highest time-span-prominence contributor (harmony follows the head — GTTM-consistent); the *slowest* blob (harmony as the background layer — Schenker-consistent); or the tension composite itself with no blob identity at all (15.iii). Candidate features → harmonic behavior: discrete triggers (merge → R move; split → L; phrase boundary → move toward home; strong boundary → arrival *on* home = cadence analog); continuous (tension → move *rate* and move *size*: calm = rare L, agitated = frequent R and PL/RP compounds; centroid height → inversion/register of the field; hue drift → major/minor balance via L:R ratio); trajectory-shaped (a vineyard point spiraling toward the diagonal → its voice resolves stepwise onto a chord tone —

voice-leading literally read off the diagram). Voice-behavior options per voice: hard snap at boundaries only; continuous gravitation (default); and *licensed dissonance* — high-prominence voices may sound non-chord tones (roughness contributors) that resolve at the next move, functionally identical to tonal dissonance treatment (suspension → resolution) without importing tonality wholesale.

15.ii Cohn/Lewin, plainly

Cohn: all 24 major/minor triads arrange into a map (the Tonnetz) where P/L/R are single steps between neighbors; "distance" between any two triads = fewest/smallest note changes to get from one to the other (parsimony). Certain repeating step-chains (e.g., P-L-P-L…) walk in closed loops of 6 or 8 triads that never establish a home chord — harmony that keeps *moving coherently* without ever cadencing. For an open-ended installation that is a feature: perpetual, logical drift without the arrival clichés that would make the piece feel like it keeps ending. Lewin: describe music by the *moves* rather than the *chords* — "we went L then R" rather than "we were on C major then E minor." That is the correct description of the entire governor layer: governors output transformations (of the clock, of patterns, of harmony), never fixed objects — one sentence of Lewin in the paper frames the whole architecture.

15.iii Harmony governed by system-level measures — yes, and it may be the best option

The Tonnetz walk needs only three inputs — *when* to step, *which* transformation, *how far* — and none requires a blob identity: **step timing** from diagram-Wasserstein between consecutive frames (topological change rate → harmonic change rate; the harmony literally moves when the topology does); **step size/type** from the tension composite (calm → L; agitated → compounds) or from surprisal (a surprising physical transition licenses a distant move — Meyer again); **direction** from the Wasserstein-to-equilibrium gradient: moving away from physical equilibrium → steps away from the home triad, returning → steps along the shortest parsimonious path home, making "distance from tonic" and "distance from rest" one governed quantity (the Q4 isomorphism, implemented); **chord cardinality/ambiguity** from conditional entropy (certain → clean triad; uncertain → added tones and cluster shading, raising roughness); statistical expectation of the next macrostate can even *pre-sound* the predicted next triad faintly (Imagination phase, Q2). Blob-identity governance (15.i) and system-level governance compose cleanly: identity features supply discrete triggers, system measures supply the continuous field — and since all of it arrives via `ScoreState` and routing sources, the balance between them stays performable from the EDIT surface.

The through-line

Q2, Q3.ii, Q7, Q10, Q11, and Q12 all reduced to *the same two mechanisms* — a symbolized-state expectation model (surprisal/entropy/hazard) and a preference-rule/beam scorer with switching costs — applied at different layers. That convergence is the architecture: `score_governor.py` needs exactly those two engines plus the corrected merge tree, and everything else is bindings. Fix the GUDHI dimension order and the three merge-tree gaps first (the attached suite reports when they are done), then the governor work stands on validated ground.

Appendix — Metric → Lamp Dynamics → Score Grammar → Codebase Mapping

Comprehensive mapping of prospective measures to the lamp elements that produce them, the score-grammar roles they could govern, and where each lands in the current project files. Integration column references the actual v4.0 tree: `blob_tracker.py` (per-blob state, Kalman, flow), `blob_detection.py` (GUDHI, watershed), `merge_tree.py`, `voice_manager.py` (`_compute_targets`, `SynthState`), `routing.py` (Source/Dest enums), `controller.py` (EDIT surface), `blob_synth.scd`; new modules `score_governor.py` / `system_state.py` are the additive homes per guideline 1.i.

Measure / notion	Lamp element & dynamics measured	Prospective score-grammar association	Incorporation into current codebase
PER-BLOB KINEMATIC & ENERGETIC			
Velocity / speed / acceleration	Individual wax blob motion; convection drive per blob	Voice modulation (existing VX/VY/SPEED/ACCEL routings); vertical sign-changes as tempo-drift evidence for the metrical module	Already tracked in `blob_tracker.py` Kalman state; expose sign-change events to `score_governor.py` meter module; existing `routing.py` sources unchanged
Kinetic energy (per blob) & energy integral	½·area·speed²; work accumulated over the blob's trajectory	Time-span reduction: energy integral as an importance term (Q9e); per-voice DRIVE/amp	Accumulate in tracker per track ID; term in prominence score inside `score_governor.py`; routed via new PROMINENCE_RANK source in `routing.py`
Integrated persistence (lifetime sum)	Blob's H0 persistence summed over its whole life — sustained vs. flash significance	Time-span reduction: age-weighted head/ornament ranking robust to short lifecycles	Running sum keyed by track ID in `blob_tracker.py`; consumed by prominence ranker; mixer weights applied in `voice_manager._compute_targets`
Path length / trajectory curvature	Total distance traveled; how convoluted the blob's route is	Ornamentation level: curvy blobs receive more diminution (pattern density, WAVE_POS movement)	Derived in tracker from position history; bind as governor-derived routing source; maps to pattern-layer `fast/ply` transforms
Elongation / d_area oscillation	Blob geometry breathing as it rises/deforms	Behavioral meter evidence (geometry periodicity enters the Q11 candidate bank); SHIMMER/FMODE modulation (existing)	Existing ELONGATION/D_AREA sources; add autocorrelation over their history in the meter module's signal bank
ENTROPY & INFORMATION MEASURES			
Permutation entropy (per signal)	Ordinal-pattern disorder of any short time series (blob velocity, KE, flux)	Erratic-role election (tension governor = flattest/most disordered signal); tension composite term	Small utility in new `system_state.py`; runs on rolling deques already kept by tracker; no changes to detection/audio paths
Persistence entropy	Shannon entropy of normalized H0 lifetimes — is significance concentrated or spread?	Texture distribution: low → soloist+accompaniment; high → even polyphony; term in tension composite	One-liner over the diagram already produced in `blob_detection.py`; emitted in ScoreState; drives per-voice amp spread in `_compute_targets`
Occupancy-grid entropy	Spatial disorder of wax distribution over a coarse ROI grid	Prolongational tension term (disorder = tension); boundary evidence when it shifts regime	Compute from foreground mask (`background_reference.py` already isolates wax); member of the SystemSummary dataclass in `system_state.py`
Conditional entropy H(next|current) of symbolized macrostate	Uncertainty of the immediate future given the present system symbol	Ambiguity governor: harmonic clarity vs. cluster shading; handover softness (Q12); structural commitment strength	Online transition matrix over the symbolized SystemSummary in `score_governor.py`; TENSION/ambiguity emitted via ScoreState; chord cardinality applied at pitch-field constraint
Entropy rate of the state chain	Global unpredictability of the lamp's regime dynamics	Long-form form control: predictable epochs → firm structure; chaotic epochs → searching score	Derived from the same transition matrix; scales all governor weights globally (confidence-weighted governance)
Surprisal –log p(transition)	Instantaneous unexpectedness of the observed state transition	Meyer/Huron tension events: dissonance-roughness spikes, accents, licensed distant Tonnetz moves	Emitted per frame by the expectation model; new routing Source (e.g. SURPRISAL) in `routing.py`; mapped to NOISE/DRIVE/DETUNE dests and harmony step size

Measure / notion	Lamp element & dynamics measured	Prospective score-grammar association	Incorporation into current codebase
Transfer entropy / mutual information (blob pairs)	Directional influence: which blob's motion predicts another's	Coupling-centrality prominence (hub blobs = heads, Q9c); ecosystemic governor weighting	Pairwise over tracker histories (n≤16 → trivial); feeds role-weight softmax in `score_governor.py`
Predictive information / excess entropy	Bits the system's past carries about its future	Journal-round refinement of ambiguity/commitment control (IDyOM bridge)	Deferred; same symbolized stream, longer contexts (PPM-style); plugs into the identical ScoreState fields — no architectural change
RELATIONAL & CONFIGURATIONAL			
Pairwise distances & their variance	Blob constellation spread; clustering vs. dispersal	Grouping boundary evidence (variance regime shifts); register spread mapping	From tracker centroids per frame; BoundaryEvidence events into the grouping module; register-band constraints in ScoreState
Proximity-graph edge flips (Delaunay / k-NN)	Nearest-neighbor topology changes as blobs rearrange	Motive-level articulation boundaries (GPR3-analog: configurational change)	scipy.spatial.Delaunay over centroids in `system_state.py`; flip events → grouping scorer
Relative phase of vertical oscillation (pairs)	Phase relationship between two blobs' rise/fall cycles	Meter: phase alignment → strong-beat placement; drift → metric tension	Hilbert/zero-crossing phase per blob in the meter signal bank; parameterizes the hierarchical clock in `score_governor.py`
Kuramoto order parameter (n blobs)	Degree of synchronization of blob oscillation phases (0 = free, 1 = locked)	Metric strength: high sync → crisp grid attraction; low → elastic, projective meter (Hasty mode)	One complex mean over per-blob phases; scales quantization/elastic-pull strength applied to pattern-event timing before OSC bundling
Period ratios between blob pairs	Ratio of two blobs' convection cycle periods (e.g. 3:2)	Polyrhythm / non-isochronous meter baked into the clock (hemiola); accent profile of the bar	Ratio of autocorrelation peak lags; sets subdivision cardinalities of the clock object; London's bounds clamp the result
Collinearity / symmetry / hull-area events	Momentary geometric organization of the constellation	Weak boundary evidence; texture gestures (alignment → unison/chorale moment)	Cheap geometry in `system_state.py`; coincidence-bonus contributors in the grouping scorer; unison gesture via pitch-field collapse in ScoreState
GLOBAL PHYSICAL & ORDER PARAMETERS			
Total kinetic energy & mean vertical flux	Whole-lamp activity level; net upward/downward wax transport	Tension composite terms; tempo drift; dynamics (global amp)	Sums over tracker state; TENSION source in `routing.py`; global amp/tempo lines in ScoreState clock params
Velocity coherence (Vicsek order parameter)	Collective alignment of blob motion (ordered flow vs. turbulent)	Order↔disorder regime: coherent → homophonic texture, unified rhythm; disordered → polyphonic independence	Mean normalized velocity in `system_state.py`; scales pattern-transform coupling (shared vs. per-voice transforms) in the pattern layer
Center-of-mass height & crossings	Bulk wax elevation; convection cycle position	Register center of the whole texture; section-level grouping when COM crosses midline	From tracker areas×positions; register-band constraint in ScoreState; crossing events → boundary scorer
Distance-from-equilibrium (Wasserstein/KL of occupancy vs. frozen reference)	How far the wax distribution sits from its cold-lamp resting state	Distance-from-tonic analog (the central isomorphism): drives Tonnetz walk direction and roughness	Compare live occupancy vs. `BackgroundReference` median (already stored); gradient sign steers harmony module's P/L/R selection
Hazard estimates (merge/death in next N s)	Lifecycle risk from historical event statistics + current configuration	Boundary schedule (audible-as-coming): accelerando/crescendo ramps; pre-fade handovers (Q12)	Fit from logged lifecycle events (extend `diagnose_tracking.py` logging); hazard scalar in ScoreState drives clock tempo ramp + governance pre-fade
Lyapunov estimate / RQA (recurrence rate, determinism, laminarity)	Chaoticity and regime texture of aggregate signals	Regime-boundary evidence; long-form character (laminar epochs → sustained schema families)	Offline first (analysis notebooks), promoted to `system_state.py` if cheap enough on Pi 5; evidence source only — no dedicated consumer needed
TOPOLOGICAL (TDA / MERGE TREE)			

Measure / notion	Lamp element & dynamics measured	Prospective score-grammar association	Incorporation into current codebase
H0 persistence per blob (instantaneous)	Feature significance of each luminance peak	Persistence-as-prominence (time-span mixer): amp, pitch stability, register priority	Existing pipeline (`blob_detection.py`) after the GUDHI dimension-order fix; PROMINENCE_RANK source; mixer weights in `_compute_targets`
Persistence diagram (birth, death) coordinates	Peak brightness level and absorption level per blob	Register/duration assignment (birth level ≈ wax height → register, near-literal)	Already computed; map birth level to base-freq register offset when a voice binds in `binding_manager.py`
Diagram Wasserstein between consecutive frames	Rate of topological change of the whole field	Harmonic step timing (harmony moves when topology moves); boundary evidence on spikes	gudhi.wasserstein on stored previous diagram in `system_state.py`; harmony module step trigger; BoundaryEvidence on z-scored spikes
Vineyard trajectories (diagram-space tracks)	Per-blob paths in (birth,death) space over time; convergence predicts merges	Voice-leading: toward-diagonal → stepwise resolution/release; converging points → pre-boundary tension	The planned Hungarian-in-diagram-space identity carrier; expose per-voice trajectory features as routing sources; resolution behavior in harmony module's gravitation
Merge-tree leaf depth	Structural level of each blob in the (corrected) tree	Schenkerian level assignment: depth → background/diminution material class per voice	After `merge_tree.py` fixes (saddle nodes, essential class, cofaces localization): depth per leaf → material-class field in ScoreState; pattern layer selects density/degree pool by class
Saddle events & levels	Actual merge levels between components (true grouping boundaries)	Phrase/section boundaries with strength = absorbed branch persistence; metric modulation triggers	Promote the test-suite union-find oracle into `merge_tree.py`; saddle events → grouping scorer; strength scales global gestures
Tree shape (depth, balance, leaf count)	Hierarchical organization of the whole field	Texture archetype: balanced → even polyphony; caterpillar → melody+accompaniment; shape entropy → tension term	Simple stats over the corrected tree per frame; texture weights in ScoreState consumed by the time-span mixer
Betti-0 count (component count)	Number of distinct wax bodies above threshold	Meter cardinality evidence (3 bodies → triple grouping); density ceiling	Already implicit in blob count; meter module evidence + density-ceiling constraint in ScoreState
STATISTICAL TOOLS & CHANGE DETECTION			
BOCPD / CUSUM changepoints (any aggregate signal)	Regime changes in KE, coherence, entropy, flux	Principal grouping-boundary evidence source with posterior probabilities as soft scores	One BOCPD utility in `score_governor.py` instantiated per signal; posteriors enter the grouping scorer directly (guideline 1.ii: one mechanism, many signals)
EMD / KL / JS between snapshot summaries	"How different is now from then" over macrostates	Form-level recurrence: revisited macrostate → musical recall/reprise (schema family return)	Distance over SystemSummary history ring buffer; recall trigger re-weights the schema-succession Markov table in the pattern layer
Rolling z-scores / statistical dominance	Signal changes normalized by their own noise floor	Self-calibrating GPR3 evidence and self-calibrating role switching (no fixed margins)	Shared utility in `system_state.py` used by both the boundary scorer and the role-weight softmax
Autocorrelation / spectral peak-flatness bank	Periodicity strength of every candidate signal (blob, pair, aggregate)	Behavioral election generalized to signals: strongest peak → meter source; flattest → tension source	Uniform analyzer applied across the Q11 signal bank in the meter module; winner parameterizes the clock; weights live on the `controller.py` EDIT surface
Hazard / survival models over lifecycle events	Time-to-merge/death distributions conditioned on configuration	ITPRA Tension-phase ramps; predictive handover; boundary pre-announcement	Fit offline from event logs, evaluate online; single hazard scalar per blob in ScoreState; consumed by clock (ramps), handover (pre-fade), harmony (approach-to-home)